

Competitive Programming

Section 60: Final Capstone

Competitive Programming Course

2026-07-15

Table of contents

1	Final capstone	3
2	Capstone protocol	4
3	Model and proof discipline	5
4	Validation loop	6
5	During the contest	7
6	Exercises	8

1 Final capstone

This section is a contest simulation, not a final set of templates. The goal is to decide what a problem is asking before selecting a tool.

2 Capstone protocol

Spend the first pass on all three tasks. For each, write:

```
the largest constraint and the naive cost it rules out  
the mathematical or combinatorial object being manipulated  
one promising model and its required invariant  
one small case that can be checked manually
```

Choose an order by confidence, not by statement order. After solving one task, return to the others with fresh eyes rather than forcing a familiar technique.

3 Model and proof discipline

Before implementation, state the transformation from the original statement to your chosen representation. Prove that it loses no solutions and adds none. Then identify the base cases, boundary conventions, and the exact quantity that each loop or transition maintains.

4 Validation loop

Build a tiny oracle whenever the input size permits it. Generate small random instances, compare outputs, and shrink any mismatch to the smallest failing case. Test zero, one, repeated values, impossible conditions, and maximum numeric ranges deliberately.

5 During the contest

Keep a short written log of rejected ideas and why their complexity or invariant fails. This prevents revisiting the same dead end. Reserve time for reading the statement again, checking types, and verifying output format.

6 Exercises

Attempt A, B, and C without consulting a technique list. For each submission, keep the model, correctness argument, complexity derivation, and at least three targeted tests. The editorial is the post-attempt debrief, not the starting point.